Annex A

(normative)

Protocol Implementation Conformance Statement (PICS) proforma

A.1 Introduction[11]

The supplier of an implementation that is claimed to conform to Clause 7 shall complete the following protocol implementation conformance statement (PICS) proforma.

A completed PICS proforma is the PICS for the implementation in question. The PICS is a statement of which capabilities and options of the protocol have been implemented. A PICS is included at the end of each clause as appropriate. The PICS can be used for a variety of purposes by various parties, including the following:

a) As a checklist by the protocol implementor, to reduce the risk of failure to conform to the standard through oversight;
b) As a detailed indication of the capabilities of the implementation, stated relative to the common basis for understanding provided by the standard PICS proforma, by the supplier and acquirer, or potential acquirer, of the implementation;
c) As a basis for initially checking the possibility of interworking with another implementation by the user, or potential user, of the implementation (note that, while interworking can never be guaranteed, failure to interwork can often be predicted from incompatible PICS);
d) As the basis for selecting appropriate tests against which to assess the claim for conformance of the implementation, by a protocol tester.

A.1.1 Abbreviations and special symbols

The following symbols are used in the PICS proforma:

M	mandatory field/function
!	negation
O	optional field/function
O.<n>	optional field/function, but at least one of the group of options labeled by the same numeral <n> is required
O/<n>	optional field/function, but one and only one of the group of options labeled by the same numeral <n> is required
X	prohibited field/function
<item>:	simple-predicate condition, dependent on the support marked for <item>
<item1>*<item2>:	AND-predicate condition, the requirement must be met if both optional items are implemented
<item1>+<item2>:	OR-predicate condition, the requirement must be met if either of the optional items are implemented

[11] *Copyright release for PICS proformas:* Users of this standard may freely reproduce the PICS proforma in this subclause so that it can be used for its intended purpose and may further publish the completed PICS.

A.1.2 Instructions for completing the PICS proforma

The first part of the PICS proforma, Implementation Identification and Protocol Summary, is to be completed as indicated with the information necessary to identify fully both the supplier and the implementation.

The main part of the PICS proforma is a fixed-format questionnaire divided into subclauses, each containing a group of items. Answers to the questionnaire items are to be provided in the right-most column, either by simply marking an answer to indicate a restricted choice (usually Yes, No, or Not Applicable), or by entering a value or a set or range of values. (Note that there are some items where two or more choices from a set of possible answers can apply; all relevant choices are to be marked.)

Each item is identified by an item reference in the first column; the second column contains the question to be answered; the third column contains the reference or references to the material that specifies the item in the main body of the standard; the sixth column contains values and/or comments pertaining to the question to be answered. The remaining columns record the status of the items—whether the support is mandatory, optional, or conditional—and provide the space for the answers.

The supplier may also provide, or be required to provide, further information, categorized as either Additional Information or Exception Information. When present, each kind of further information is to be provided in a further subclause of items labeled A<i> or X<i>, respectively, for cross-referencing purposes, where <i> is any unambiguous identification for the item (e.g., simply a numeral); there are no other restrictions on its format or presentation.

A completed PICS proforma, including any Additional Information and Exception Information, is the protocol implementation conformance statement for the implementation in question.

Note that where an implementation is capable of being configured in more than one way, according to the items listed under Major Capabilities/Options, a single PICS may be able to describe all such configurations. However, the supplier has the choice of providing more than one PICS, each covering some subset of the implementation's configuration capabilities, if that would make presentation of the information easier and clearer.

A.1.3 Additional information

Items of Additional Information allow a supplier to provide further information intended to assist the interpretation of the PICS. It is not intended or expected that a large quantity will be supplied, and the PICS can be considered complete without any such information. Examples might be an outline of the ways in which a (single) implementation can be set up to operate in a variety of environments and configurations; or a brief rationale, based perhaps upon specific application needs, for the exclusion of features that, although optional, are nonetheless commonly present in implementations.

References to items of Additional Information may be entered next to any answer in the questionnaire, and may be included in items of Exception Information.

A.1.4 Exceptional information

It may occasionally happen that a supplier will wish to answer an item with mandatory or prohibited status (after any conditions have been applied) in a way that conflicts with the indicated requirement. No preprinted answer will be found in the Support column for this; instead, the supplier is required to write into the Support column an X<i> reference to an item of Exception Information, and to provide the appropriate rationale in the Exception item itself.

An implementation for which an Exception item is required in this way does not conform to this standard.

Note that a possible reason for the situation described above is that a defect in the standard has been reported, a correction for which is expected to change the requirement not met by the implementation.

A.1.5 Conditional items

The PICS proforma contains a number of conditional items. These are items for which both the applicability of the item itself, and its status if it does apply—mandatory, optional, or prohibited—are dependent upon whether or not certain other items are supported.

Individual conditional items are indicated by a conditional symbol of the form “<item>:<s>” in the Status column, where “<item>” is an item reference that appears in the first column of the table for some other item, and “<s>” is a status symbol, M (Mandatory), O (Optional), or X (Not Applicable).

If the item referred to by the conditional symbol is marked as supported, then 1) the conditional item is applicable, 2) its status is given by “<s>”, and 3) the support column is to be completed in the usual way. Otherwise, the conditional item is not relevant and the Not Applicable (N/A) answer is to be marked.

Each item whose reference is used in a conditional symbol is indicated by an asterisk in the Item column.

A.1.6 Identification

A.1.6.1 Implementation identification

Supplier (Note 1)	
Contact point for queries about the PICS (Note 1)	
Implementation Name(s) and Version(s) (Notes 1 and 3)	
Other information necessary for full identification— e.g., name(s) and version(s) of machines and/or operating system names (Note 2)	
NOTE 1—Required for all implementations. NOTE 2—May be completed as appropriate in meeting the requirements for the identification. NOTE 3—The terms Name and Version should be interpreted appropriately to correspond with a supplier’s terminology (e.g., Type, Series, Model).	

A.1.6.2 Protocol summary

Identification of protocol specification	IEEE Std 802.1CB-2017, IEEE Standard for Frame Replication and Elimination for Reliability
Identification of amendments and corrigenda to the PICS proforma that have been completed as part of the PICS	Amd : ________________ Cor: _________________ Amd : ________________ Cor: _________________
Have any exceptions been noted? (See A.1.4. The answer, “Yes” means that the implementation does not conform to IEEE Std 802.1CB.)	Yes [ ] No [ ]

A.2 PICS proforma for Frame Replication and Elimination for Reliability

A.2.1 Major capabilities/options

Item	Feature	Subclause	Value/Comment	Status	Support
BG	FRER C-component implemented?	5.15		O	Yes [ ] No [ ]
IS	Stream identification system implemented?	5.3, 5.4, 5.5	One or more of IS, TE, LE, and/or RS must be answered "Yes."	O.1	Yes [ ] No [ ]
TE	Talker end system implemented?	5.6, 5.7, 5.8		O.1	Yes [ ] No [ ]
LE	Listener end system implemented?	5.9, 5.10, 5.11		O.1	Yes [ ] No [ ]
RS	Relay system implemented?	5.12, 5.13, 5.14, 5.15:b, 5.15:c		BG:M + O.1	Yes [ ] No [ ]

A.2.2 Stream identification component

Item	Feature	Subclause	Value/Comment	Status	Support
IS1	Can the system identify frames using the Null Stream identification function?	5.3:b, 6.4		IS: M	Yes [ ]
IS2	Does the system implement the required managed objects of Clause 9?	5.3:c, 9		IS: M	Yes [ ]
IS3	Can the system encode frames using the Active Destination MAC and VLAN Stream identification?	5.4:a, 6.6		IS: O	Yes [ ] No [ ] ___[a]
IS4	Can the system identify packets using the IP Stream identification?	5.5:c, 6.7		IS: O	Yes [ ] No [ ]
IS5	For what additional Stream decodings can the system be configured?	5.5:d		IS: O	—
IS6	Explain the limits on which ports the above features can be configured.	5.5:a		IS: O	—
IS7	Explain the limits on the number of Streams for which the above features can be configured.	5.5:b		IS: O	—

[a]If "No," supply a reason why.

A.2.3 Talker end system

Item	Feature	Subclause	Value/Comment	Status	Support
TE8	Can the system identify frames using the Null Stream identification function?	5.6:b, 6.4		TE: M	Yes [ ]
TE9	Can the system be configured with a Sequence generation function?	5.6:c, 7.4.1		TE: M	Yes [ ]
TE10	Can the system be configured with a Sequence encode/decode function?	5.6:d, 7.8		TE: M	Yes [ ]
TE11	Does the system implement the managed objects of Clause 9 and Clause 10 (10.7 not required)?	5.6:e, 9, 10		TE: M	Yes [ ]
TE12	Can the system encode frames using the Active Destination MAC and VLAN Stream identification?	5.7:a, 6.6		TE: O	Yes [ ] No [ ] ___[a]
TE13	Can the system be configured with a Stream splitting function?	5.7:b, 7.7		TE: M	Yes [ ] No [ ] ___[a]
TE14	Can the system identify packets using the IP Stream identification?	5.8:c, 6.7		TE: O	Yes [ ] No [ ]
TE15	For what additional Stream decodings can the system be configured?	5.8:d		TE: O	—
TE16	Can the system encode frames using HSR sequence tag?	5.8:e, 7.9		TE: O	Yes [ ] No [ ]
TE17	Can the system encode frames using PRP sequence trailer?	5.8:f, 7.10		TE: O	Yes [ ] No [ ]
TE18	For what additional Sequence encode/decode functions can the system be configured?	5.8:g		TE: O	—
TE19	Explain the limits on which ports the above features can be configured.	5.8:a		TE: O	—
TE20	Explain the limits on the number of Streams for which the above features can be configured.	5.8:b		TE: O	—

[a]If "No," supply a reason why.

A.2.4 Listener end system

Item	Feature	Subclause	Value/Comment	Status	Support
LE1	Can the system identify frames using the Null Stream identification?	5.9:b, 6.4		LE: M	Yes [ ]
LE2	Can the system be configured with at least two Individual recovery functions?	5.9:c, 7.5		LE: M	Yes [ ]
LE3	Can the system be configured with at least one Sequence recovery function using the MatchRecoveryAlgorithm?	5.9:c, 7.4.2, 7.4.3.5		LE: M	Yes [ ]
LE4	Does the system support the Sequence recovery function using the VectorRecoveryAlgorithm with a value of frerSeqRcvyHistoryLength ≥ 2?	5.9:c, 7.4.2, 7.4.3.4		LE: M	Yes [ ]
LE5	Can the system be configured with at least two Individual recovery functions using the MatchRecoveryAlgorithm?	5.9:d, 7.5, 7.4.3.5		LE: M	Yes [ ]
LE6	Can the system be configured with a Sequence decoding function?	5.9:e, 7.8		LE: M	Yes [ ]
LE7	Does the system implement the managed objects of Clause 9 and Clause 10 (10.7 not required)?	5.9:f, 9, 10		LE: M	Yes [ ]
LE8	Does the Base recovery function process a frame before its FCS has been verified?	7.4.3		LE: M	No [ ]
LE9	Can the system decode frames using the Active Destination MAC and VLAN Stream identification?	5.10:a, 6.6		LE: O	Yes [ ] No [ ] ___[a]
LE10	Can the system decode packets using the IP Stream identification?	5.11:c, 6.7		LE: O	Yes [ ] No [ ]
LE11	For what additional Stream decodings can the system be configured?	5.11:d		LE: O	—
LE12	Can the system decode frames using HSR sequence tag?	5.11:e, 7.9		LE: O	Yes [ ] No [ ]
LE13	Can the system decode frames using PRP sequence trailer?	5.11:f, 7.10		LE: O	Yes [ ] No [ ]
LE14	For what additional Sequence decodings can the system be configured?	5.11:g		LE: O	—

Item	Feature	Subclause	Value/Comment	Status	Support
LE15	Can the system be configured with at least two Individual recovery functions using the VectorRecoveryAlgorithm?	5.11:h, 7.5, 7.4.3.4		LE: O	Yes [ ]
LE16	Explain the limits on which ports the above features can be configured.	5.11:a		LE: O	—
LE17	Explain the limits on the number of Streams for which the above features can be configured.	5.11:b		LE: O	—

[a]If "No," supply a reason why.

A.2.5 Relay system

Item	Feature	Subclause	Value/Comment	Status	Support
RS1	Can the system identify frames using the Null Stream identification function?	5.12:b, 6.4		RS: M	Yes [ ]
RS2	Can the system be configured with a Sequence generation function?	5.12:c, 7.4.1		RS: M	Yes [ ]
RS3	Can the system be configured with at least two Individual recovery functions?	5.12:e, 7.5		RS: M	Yes [ ]
RS4	Can the system be configured with at least one Sequence recovery function using the MatchRecoveryAlgorithm?	5.12:e, 7.4.2, 7.4.3.5		RS: M	Yes [ ]
RS5	Does the system support the Sequence recovery function using the VectorRecoveryAlgorithm with a value of frerSeqRcvyHistoryLength $\geq$ 2?	5.12:e, 7.4.2, 7.4.3.4		RS: M	Yes [ ]
RS6	Can the system be configured with at least two Individual recovery functions using the MatchRecoveryAlgorithm?	5.12:f, 7.5, 7.4.3.5		RS: M	Yes [ ]
RS7	Can the system be configured with a Sequence encode/decode function?	5.12:d, 7.8		RS: M	Yes [ ]
RS8	Does the system implement the managed objects of Clause 9 and Clause 10 (including 10.7)?	5.12:g, 9, 10		RS: M	Yes [ ]
RS9	Does the Base recovery function process a frame before its FCS has been verified?	7.4.3		RS: M	No [ ]

Item	Feature	Subclause	Value/Comment	Status	Support
RS10	Can the system encode/decode frames using the Active Destination MAC and VLAN Stream identification?	5.13:a, 6.6		RS: O	Yes [ ] No [ ] ___[a]
RS11	Can the system identify packets using the IP Stream identification?	5.13:b, 6.7		RS: O	Yes [ ] No [ ] ___[a]
RS12	For what additional Stream identification functions can the system be configured?	5.14:c		RS: O	—
RS13	Can the Stream splitting function be configured on the system?	5.14:d, 7.7		RS: O	Yes [ ] No [ ]
RS14	Can the system encode/decode frames using HSR sequence tag?	5.14:e, 7.9		RS: O	Yes [ ] No [ ]
RS15	Can the system encode/decode frames using PRP sequence trailer?	5.14:f, 7.10		RS: O	Yes [ ] No [ ]
RS16	For what additional Sequence encode/decode functions can the system be configured?	5.14:g		RS: O	—
RS17	Can the system be configured with at least two Individual recovery functions using the VectorRecoveryAlgorithm?	5.14:i, 7.5, 7.4.3.4		RS: O	Yes [ ] No [ ]
RS18	Can the system be configured for Autoconfiguration via the Managed objects for autoconfiguration?	5.14:j, 7.11, 10.7		RS: O	Yes [ ] No [ ]
RS19	Explain the limits on which ports the above features can be configured.	5.14:a		RS: O	—
RS20	Explain the limits on the number of Streams for which the above features can be configured.	5.14:b		RS: O	—
RS21	Explain the limits on whether the above features can be configured at in-facing or out-facing positions.	5.14:h		RS: O	—

[a]If “No,” supply a reason why.

A.2.6 FRER 802.1Q C-component

Item	Feature	Subclause	Value/Comment	Status	Support
CB1	Does the FRER 802.1Q C-component conform to the required and optional behaviors required of an IEEE 802.1Q C-VLAN component?	5.15:a		BG:M	Yes [ ]
CB1	Does the FRER 802.1Q C-component conform to the placement of the FRER functions of Clause 8?	5.15:d, 8		BG:M	Yes [ ]
CB2	Does the FRER 802.1Q C-component implement all required managed objects?	8.4		BG:M	Yes [ ]

A.2.7 Common requirements

Item	Feature	Subclause	Value/Comment	Status	Support
COM1	Does the R-TAG use the specified EtherType?	7.8.1		M	Yes [ ]
COM1	Is the Reserved field of the R-TAG transmitted as 0 and ignored on receipt?	7.1.1:d		M	Yes [ ]
COM2	Do all managed object counters roll over to 0 from their maximum value?	10.1		M	Yes [ ]
COM3	Can the system be configured with a latent error detection function?	7.4.4	Latent error detection required if sequence recovery supported	LE+RS: M	N/A [ ] Y [ ]
COM4	Is the minimum value for frerSeqRcvyLatentErrorPeriod no larger than 1 s?	10.4.1.12.2		LE+RS: M	N/A [ ] Y [ ]
COM5	Is the RemainingTicks decremented at least 100 ticks/s?	7.4.3.2.5		LE+RS: M	N/A [ ] Y [ ]
COM6	Does the system return an error if an attempt is made to configure conflicting requirements?	10		M	Yes [ ]
COM7	Is the minimum supported value of frerSeqRcvyLatentResetPeriod no larger than 1 s?	10.4.1.12.4		LE+RS: M	N/A [ ] Y [ ]
COM8	Are all counters 64 bits in length on links faster than 650 000 000 bits/s?	10.8, 10.9		M	N/A [ ] Y [ ]

Annex B

(informative)

Interoperability with other standards

IEC 62439-3 defines High-availability Seamless Redundancy (HSR) and the Parallel Redundancy Protocol (PRP). IEEE 802.1CB Frame Replication and Elimination for Reliability (FRER) has features that enable, though the procedures are not specified by this standard, to achieve a degree of interoperation between systems employing IEEE Std 802.1CB, and systems employing some other, similar protocol, including HSR and PRP, and RFC 3985 pseudowires (see [B4]).

B.1 Sequence number size

HSR, PRP, and pseudowires all use 16-bit sequence numbers. This standard supports the same sized sequence_number subparameter, in order to be maximally compatible.

B.2 Per-Stream versus per-source sequencing

IEC 62439-3 HSR/PRP uses one variable per source MAC address to sequence Ethernet frames. That is, frames belonging to different flows, and sent to different destinations, all share a single sequence number space. FRER, as defined in this standard, can be configured in the same way, or can be configured to generate a separate sequence number space for each Stream. For interoperability with HSR or PRP, per-source sequence numbering should be configured by the user.

A problem needs to be considered if one configures a VectorRecoveryAlgorithm (7.4.3.4) to receive Intermittent Streams (item c in 7.1.1) transmitted by an HSR/PRP end system. The HSR/PRP end system uses a single sequence number, not one per Stream. If that end system is transmitting more than one Stream, and some Listener is receiving less than all of those Streams, then that Listener can observe gaps in the sequence_number subparameters of the received packets, which will be counted in frerCpsSeqRcvyLostPackets (10.8.7). For both Intermittent Streams and Bulk Streams, the Base recovery function's frerSeqRcvyHistoryLength managed object (10.4.1.6) has to be large enough to accommodate both the delivery time difference and the maximum possible gap due to packets sent to other destinations, because packets received outside the window defined by frerSeqRcvyHistoryLength are discarded. If frerSeqRcvyHistoryLength is not large enough, duplicate packets will be delivered.

Annex C

(informative)

Frame Replication and Elimination for Reliability in systems

The building blocks described in Clause 6 and Clause 7 can be put together in many ways to achieve particular goals. A few examples are given in this annex. These examples are chosen, rather arbitrarily, to illustrate the range of applicability of FRER; they are not meant to constrain its use or to describe the only way to accomplish any given goal.

C.1 Example 1: End-to-end FRER

Illustrated in Figure C-1 is a simple network utilizing FRER for a single Compound Stream. In this example, all FRER functions are confined to the two end systems B and G. “Split” indicates that the Stream splitting function (7.7) is acting on transmitted packets. “Seq.” and “Rec.” indicate whether the Sequencing function is acting on transmitted packets (Sequence generation function, 7.4.1) or received packets (Sequence recovery function, 7.4.1, 7.4.2).

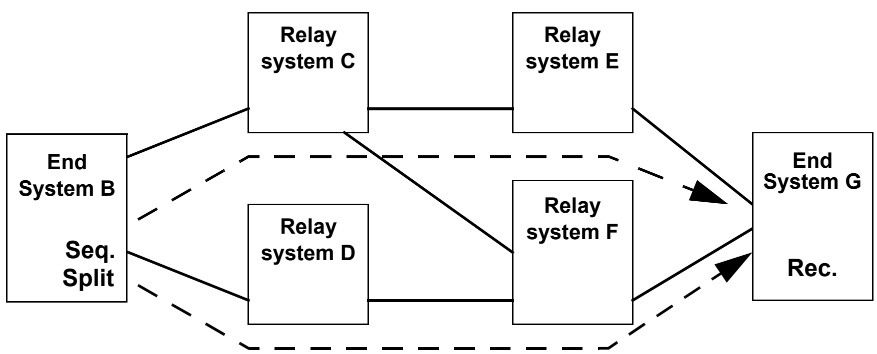

Figure C-1—Dual-homed end systems using Link Aggregation

In this example, both end systems utilize IEEE 802.1AX Link Aggregation along with FRER in order to connect their two ports (each) to a single protocol stack, as shown in Figure C-2 (end system B) and Figure C-3 (end system G). In end system B, each packet is replicated and given two different stream_handle subparameter values. The two stream_handles result in the two packets being assigned two different VLAN IDs. Based on those VLAN IDs, the two packets are both transmitted, on different physical ports, by Link Aggregation. The Distributed Resilient Network Interconnect (DRNI) feature of IEEE Std 802.1AX-2014 allows each end system’s aggregation to terminate in two relay systems.

In this example, the two-port end systems never relay packets from one port to the other; they are strictly end systems, and never act as relay systems. See C.2 and C.6 for similar examples that use alternative methods for building an end system.

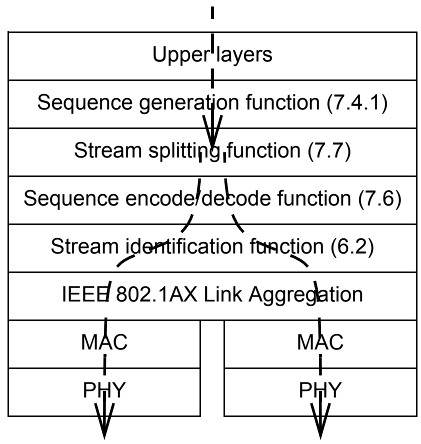

Figure C-2—Protocol stack for End System B in Figure C-1

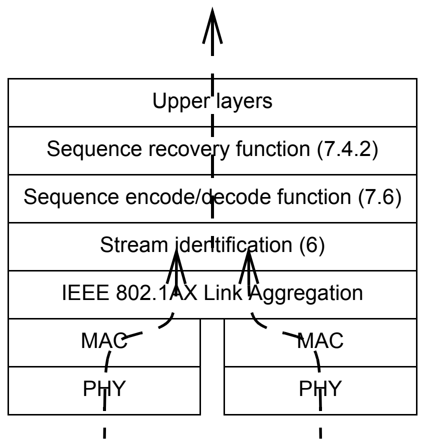

Figure C-3—Protocol stack for End System G in Figure C-1 and Figure C-4

C.2 Example 2: Various stack positions

Illustrated in Figure C-4 is a simple network utilizing FRER for a single Compound Stream, in which a relay system serves as a proxy for an end system that has no FRER capability (see item h in 7.1.1). In this figure, the numbers (1, 2, 3) are points in item e in 7.1.1. “Split” indicates that the Compound Stream is split into two streams by the ordinary bridge multicast mechanism, with the {VLAN, destination address} altered by Stream identification functions on output. [No Stream splitting function (7.7) is used.] “Seq.” and “Rec.” indicate whether the Sequencing function is acting on transmitted packets (sequence generation) or received packets (sequence recovery).

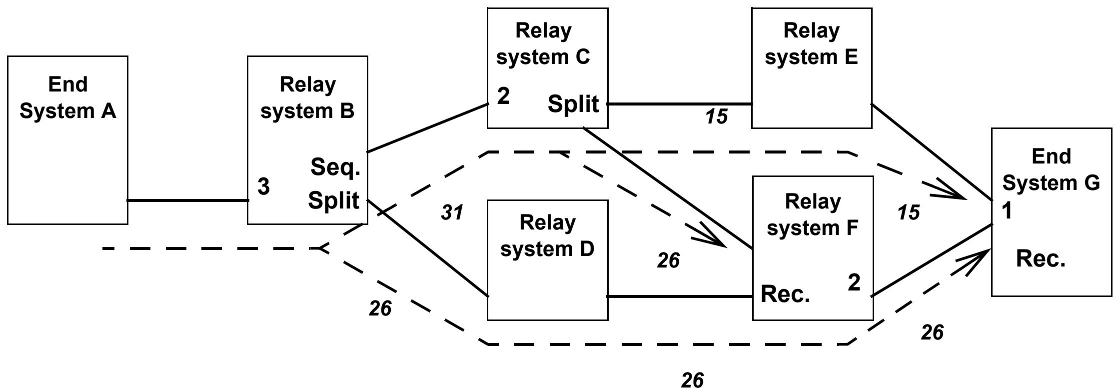

Figure C-4—Frame Replication and Elimination for Reliability flexible positioning

In Figure C-4, End System A is transmitting a Stream, but has no FRER functions. Relay system B transforms that Stream into a Compound Stream by sequencing the packets and splitting them into Member Streams *26* and *31* to go to relay systems C and D. Relay system C further splits the Stream into Streams *15* and *26* to go to relay systems E and F. (No sequencing is required; the packets were sequenced by relay system B.) Relay system F combines the two Member Streams *26* from relay systems C and D, outputting a single copy of each Packet on Stream *26* to End System G. End System G merges the two remaining Member Streams *15* and *26*, and discards the extras.

NOTE—In this example, stream_handle values are shown on the wires, being relayed from system to system. This is for the convenience of the example. In an actual network, the stream_handle is an arbitrary integer that has meaning only within a system, and is mapped to or from an external representation by the Stream identification functions (6.2) in the various systems. That external representation can, but does not necessarily, include an explicit field corresponding to a stream_handle subparameter.

Figure C-5 illustrates relay system B in Figure C-4. As the packets enter from the left, from End System A, they pass first through a Stream identification function [IP Stream identification (6.7)], which identifies the Stream. The Stream Transfer Function delivers the packet with all TSN parameters, including the stream_handle subparameter, to the Sequence generation function (7.4.1, marked “Seq.” in Figure C-4), which adds a sequence_number subparameter with a steadily-increasing integer sequence value (modulo the size of the packet field carrying the sequence_number). The sequence_number subparameter is encapsulated into the packet by the Sequence encode/decode function (7.6). A Stream identification function [this time, Active Destination MAC and VLAN Stream identification (6.6)] modifies the two packets’ destination MAC addresses and VLANs for identification through the bridged network. Relay system B’s forwarding function then outputs the two packets on two different ports. The external form of the packets are labeled differently, as indicated by the italic numbers *26* and *31* in Figure C-4.

Figure C-6 illustrates relay system C in Figure C-4. The packets on Stream *31* enter from the left, from relay system B. On output, they pass first through a passive Stream identification (Clause 6) function, which identifies the Stream. The Stream Transfer Function delivers the packet with all TSN parameters, including the stream_handle subparameter, to an active Stream identification (Clause 6) function. No Sequencing function, Sequence generation function, or Sequence recovery functions are needed, because the packets have already been sequenced (by relay system B) and none are being discarded. Stream identification encapsulates the two packets differently on the two output ports (only one output port is shown in Figure C-6), marking them as belonging to Streams *15* and *26*, and relay system C’s forwarding function outputs the two packets on two different ports.

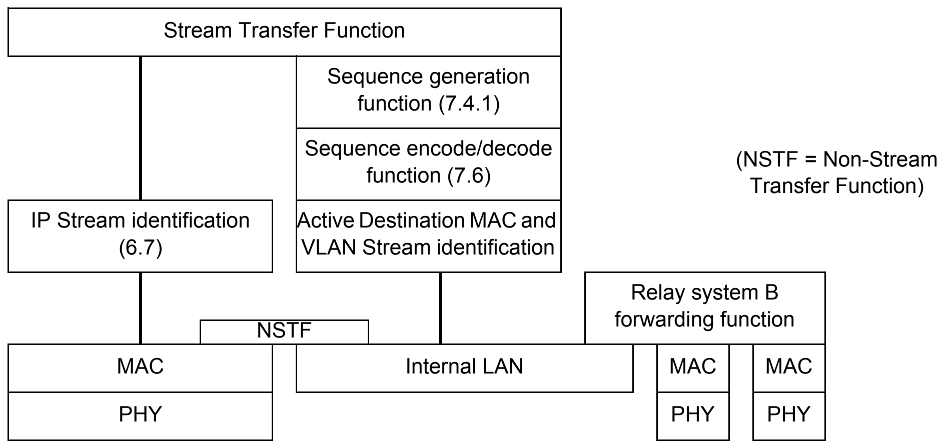

Figure C-5—Protocol stack for relay system B, proxying for End System A, in Figure C-4

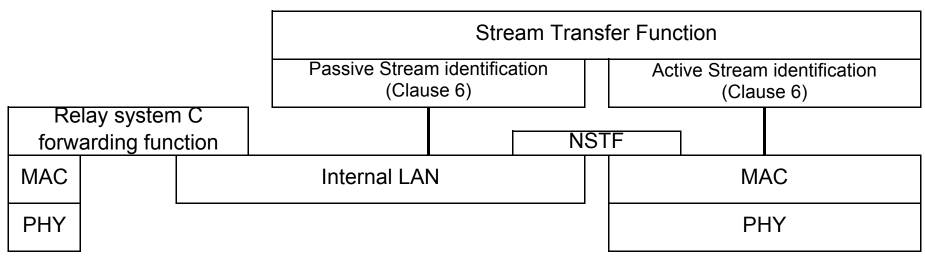

Figure C-6—Protocol stack for relay system C in Figure C-4

Figure C-7 illustrates relay system F in Figure C-4. In this case, the TSN layers are on the output port, rather than the input port, as was the case for relay system B. Data packets from both Streams *26* are brought together simply by the fact that they are both output to the same port. On that port, the Stream identification function gives them both the same stream_handle subparameter, effectively merging them into a single Stream. A Sequence encode/decode function (7.6) extracts the sequence_number subparameters from the packets, so that the Sequence recovery function (7.4.2) can discard the replicates. The Stream Transfer Function delivers all of these parameters to a Sequencing function and a Stream identification function that re-encapsulate the packets, marking them all as belonging to Stream *26*, the same as when they were received. In this particular example, unlike Figure C-5, no translation of Stream identification functions is being performed.

Finally, the protocol stack in the right-hand End System G of Figure C-4 is shown in Figure C-3. We assume in this example that relay systems E and F use IEEE 802.1AX Distributed Resilient Network Interface (DRNI), and that End System G uses IEEE 802.1AX Link Aggregation, in order to make End System G's two physical ports appear, to the network, to be a single port, even though they are connected to two different relay systems. The packets on Streams *15* and *26* are brought together by Link Aggregation. The Stream identification function assigns the same stream_handle subparameter to packets of both Streams, effectively merging them into a single Stream. They then pass through the Sequence recovery function, which deletes the replicates for delivery to the upper layers. See Figure C-2 for a similar example of a Talker, and C.6 for another type of two-port end system.

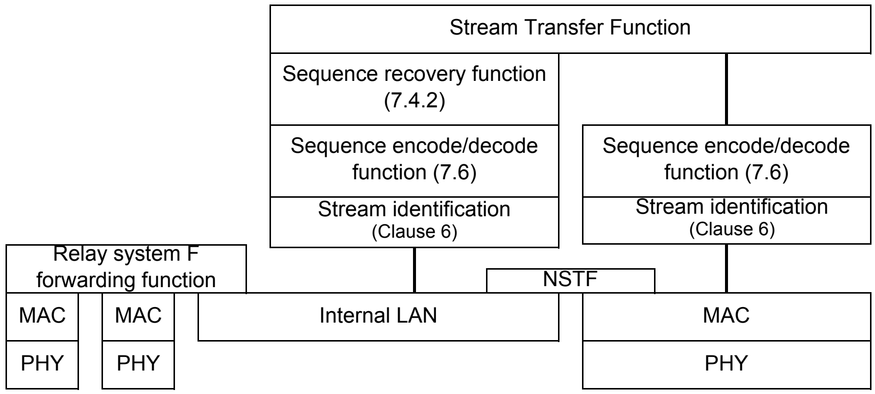

Figure C-7—Protocol stack for relay system F in Figure C-4

C.3 Example 3: Ladder redundancy

Illustrated in Figure C-8 is a network implementing "ladder redundancy." This network will continue to function in the face of multiple failures, because the Stream is repeatedly split and remerged.

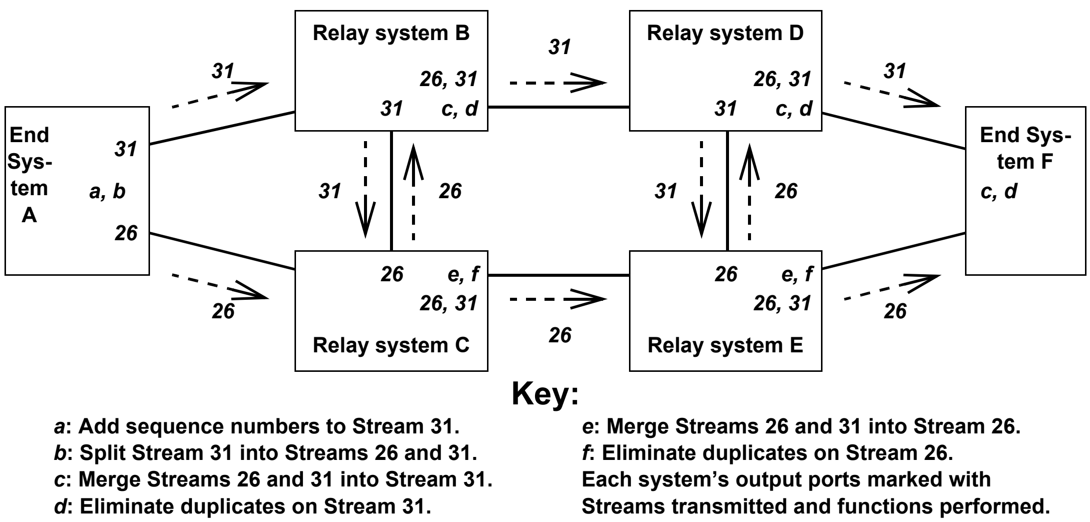

Figure C-8—Ladder redundancy

In Figure C-8, the Talker end system sequences Stream 31, then splits it into two Streams 26 and 31, sending one on each of its two ports. The network "ladder" has two "rails," an upper (relay systems B and D) and a lower (relay systems C and E). The "rungs" are the connections B–C and D–E. Each relay system sends the Stream received on the rail from the left both to the right, along the rail, and up or down, over the rung. It forwards the packets received from the rung only to the right, along the rail. At each output port to the rail, the end systems have a Sequence recovery function (7.4.2) to eliminate duplicates.

Each of the relay systems in Figure C-8 uses the protocol stack illustrated in Figure C-7.

C.4 Example 4: Multicast trees

Illustrated in Figure C-9 is a multicast FRER application. Each Stream is a multicast Stream. There are two paths to get from the Talker to each Listener; no single failure can disrupt both paths to any relay system. Presumably, each relay system has a Sequence recovery function (7.4.2) on each port to each Listener, so that each Listener receives only one copy of each packet of the Stream.

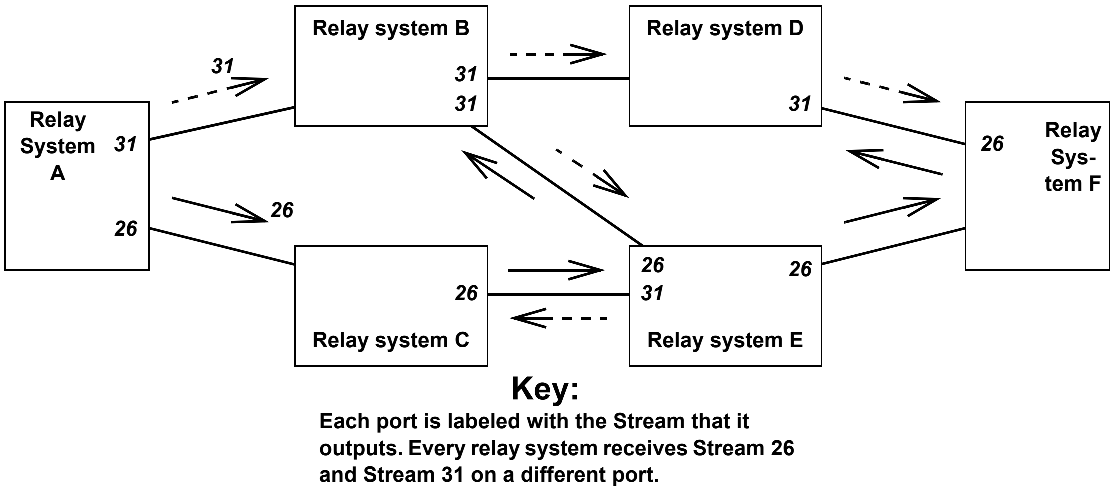

Figure C-9—Multicast trees

NOTE—The reader may find it informative to see that not all packets are replicated on all links in Figure C-9. It is not necessary to use each link twice; it is only necessary to get each packet to each relay system twice.

C.5 Example 5: Protocol interworking

Figure C-10 illustrates a simple protocol interworking function in one port of a relay system. In this example, two different encapsulation schemes **1** and **2** are used for the two legs of the Stream Transfer Function, so that packets are transformed from using one encapsulation to using the other encapsulation as they pass through the port. No additional functions, e.g., a Sequence recovery function (7.4.2) are shown, although they would be perfectly admissible. If this were a port of a bridge attached to an end system, encapsulation **1** could be the Active Destination MAC and VLAN Stream identification (6.6), and encapsulation **2** could be the IP Stream identification (6.7). The net result for the end system could be to convert a specific unicast IP Stream to use a specific multicast destination address and VLAN, in order to direct the packet through a specific path through the bridged network. Presumably, a similar interworking pair at the other end of the Stream would restore the packet to its original destination MAC address and VLAN.

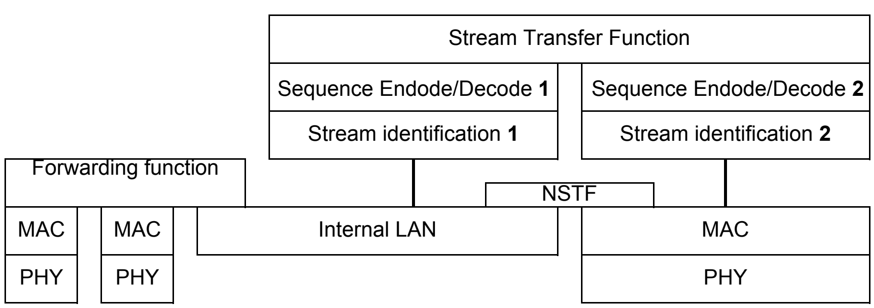

Figure C-10—Protocol interworking

C.6 Example 6: Chained two-port end systems

Illustrated in Figure C-11 is a simple network utilizing FRER for a single Compound Stream. In this example, all FRER functions are confined to the two end systems B and G. “Seq.” and “Rec.” indicate whether the Sequencing function is acting on transmitted packets (sequence generation) or received packets (sequence recovery). In this example, unlike that in C.1, both end systems are composite systems; each uses a separate one-port end system (B1, G1) attached to a three-port FRER C-component (B2, G2) in order to connect their two ports (each) to a single protocol stack, as shown in Figure C-12.

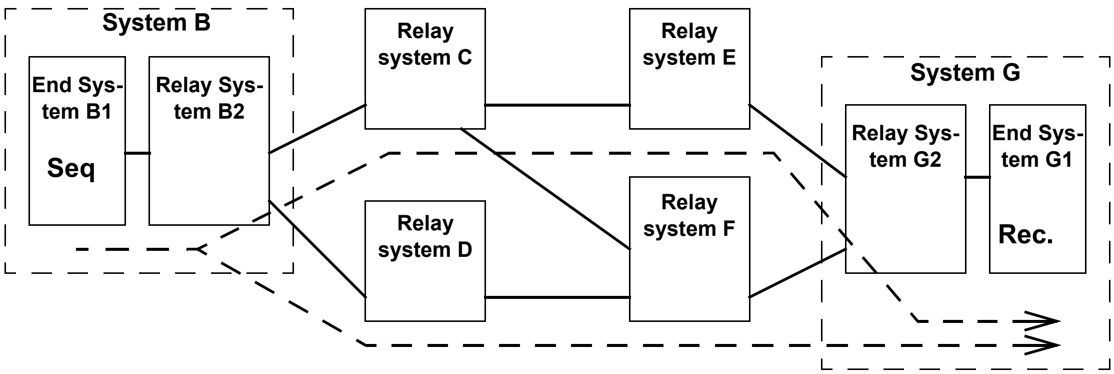

Figure C-11—Dual-homed end systems using 3-port bridge

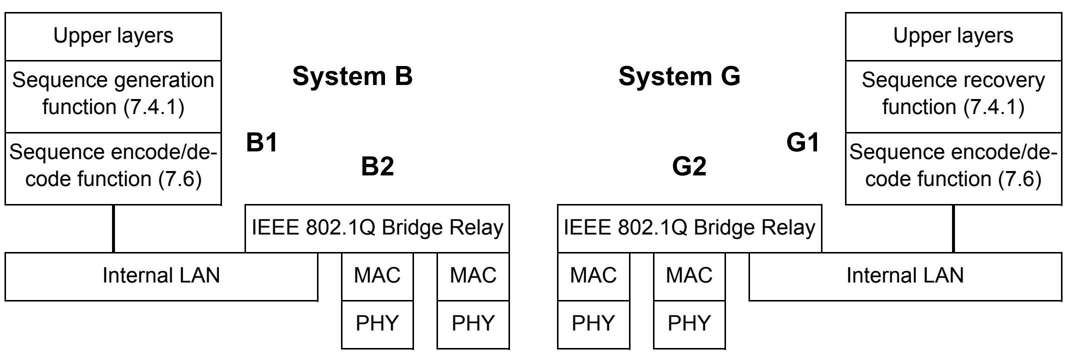

Figure C-12—Protocol stacks for Systems B and G in Figure C-11

In this example, we assume that the relay systems are all FRER C-components, and that the network has been configured so that:

a) The Stream packets carry a multicast destination MAC address;
b) Bridge B2 has been configured to transmit the (multicast) Member Stream on both ports; and
c) The other Bridges, especially G2, have been configured to allow both Member Streams to enter Bridge G2, but that Bridge G2 passes the Stream to end system G1 only (that is, the normal rules for a multicast tree are broken).

Given this configuration, no Stream splitting function (7.7) is necessary for end system B1; packet duplication is handled by the multicast forwarding functions performed in its associated relay system B2.

Unlike the example in C.1, the two-port composite end systems’ Bridge functions (B2, G2) are required to play their part in the network as Bridges, operating protocols and forwarding packets.

C.7 Cautions

Explicit path creation carries a clear danger when combined with automatic network forwarding, e.g., as defined by IEEE Std 802.1Q. Figure C-13 shows an explicit tree (the dashed arrows) established in relay system D that sends a particular Stream to relay system B and end system F. (Perhaps, at one time, the Talker was attached to relay system D and the Listeners were towards B and F.) Now, end system A starts transmitting that Stream along an IEEE 802.1Q spanning tree. When the Stream hits relay system D, it is directed back to relay system B, and the Stream loops, multiplying the packets until the bandwidth is saturated. Note also that, if this is a bridged network, relay system B (a bridge) learns conflicting information about the path to the Stream's source; System A's source MAC address is being received both from end system A and relay system D.

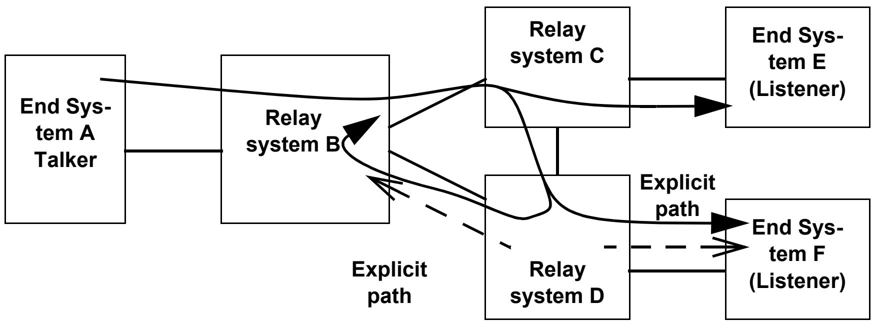

Figure C-13—Explicit path causing a loop

In order to avoid this situation in a bridged network, the application or individual that sets up the explicit paths can configure them so that they are kept, end-to-end, on separate VLANs from traffic that is not following explicit paths. On the explicit path VLANs, source learning is not performed, and destination addresses not found in the filtering database are discarded, not flooded.

C.8 Balancing tag insertion and removal

A Sequence encode/decode function will often add something to a packet. The R-TAG described in 7.8, for example, adds six octets. In a network in which not all end systems contain a Sequence encode/decode function (7.6), instances of that function can be configured in relay systems adjacent to the end systems lacking the layer, in order to remove the extra information (e.g., the R-TAG) from the packets before delivery to the end system. If the tag is not removed, and the end system has no Sequence encode/decode function, the end system will be unable to parse the received packet. It is up to the network administrator and/or configuration and management software to make sure that this does not happen.

C.9 FRER and reserved bandwidth

Satisfying the Zero congestion loss goal (item k in 7.1.1) can require the forwarding of packets to be delayed. The problem is that the different paths taken by packets in a Compound Stream can take different times to reach various Sequence recovery functions (7.4.2). If these times are very different, and if the faster path fails and then recovers, the receiving Sequence recovery function can be presented with a double-rate load of packets that all have to be delivered.

As an example, Figure C-14 illustrates a network with two Member Streams being merged into a single Stream by a Sequence recovery function. The two paths are of different lengths such that, typically, there are 40 more packets in flight on one path than on the other.

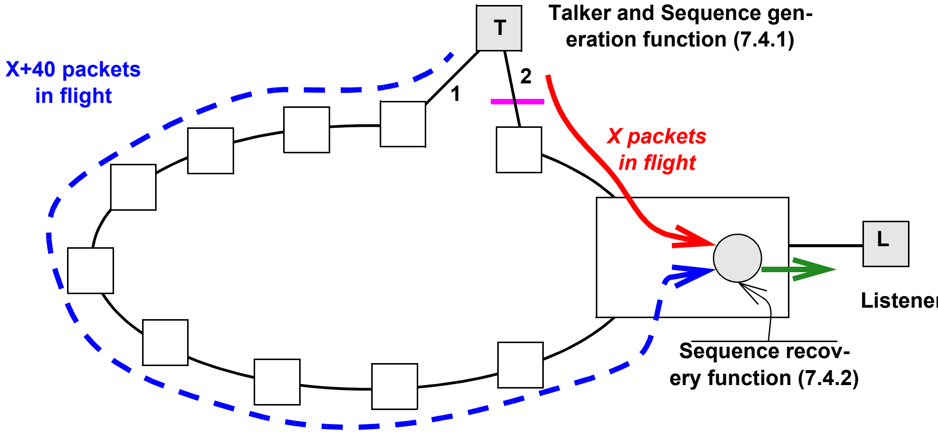

Figure C-14—Example of Long and short paths

Let us examine a hypothetical sequence of events as Link 2 in Figure C-14 fails, and then recovers. In the following list, the sequence numbers of packets taking the path through Link 2 (the short path) are in *italics*, and packets that are discarded by the Sequence recovery function are marked with asterisks (*).

a) The network is in a steady state. The Sequence recovery function receives packets:
1040, 1000*, *1041*, 1001*, *1042*, 1002*, …
That is, excepting the occasional random packet loss (not shown), the packets taking the *short path* (Link 2) are passed on, and those taking the long path (Link 1) are discarded.

b) Link 2 fails sometime later (4000 packets later). The Sequence recovery function receives:
5040, 5000* (failure occurs), 5001*, 5002*, …
That is, for a while at least, no packets are passed on; those in the slow path have already been seen.

c) The discards continue with no packets passed until the backlog on the slow path is emptied:
5038*, 5039*, 5040*, 5041, 5042, 5043, …
Normal output rates then resume, with all packets coming from the long path (Link 1).

d) When the *short path* (Link 2) heals (3000 packets later), the Sequence recovery function receives:
7998, 7999, (link heals) *8040*, 8000, *8041*, 8001, *8042*, 8002, …
Note that the Sequence recovery function is now passing packets at twice the normal rate, until the 40-packet backlog on the long (Link 1) path has been passed along.

e) Finally, the backlog is exhausted, and the Sequence recovery function resumes the normal output rate, passing packets from the *short path* (Link 2) and discarding those from the long path (Link 1):
8078, 8038, *8079*, 8039, *8080*, 8040*, *8081*, 8041*, *8082*, 8042*, …

Zero congestion loss goal (item k in 7.1.1) can require that packets be output to the Listener at a fixed maximum rate over some observation interval smaller than the transmission time of this 80-packet burst. (See, for example, Clauses 34 and 35 of IEEE Std 802.1Q-2014.) In that case, some means of buffering, and thus delaying the packets in this burst, is required at or near the Sequence recovery function.

This buffering and/or delaying is beyond the scope of this standard. We can observe, however, that the extra buffering required by FRER in order to prevent congestion loss when bandwidth reservation is employed, due to this problem, can be calculated separately from the buffering required for the non-FRER case, based on the bandwidth of the Compound Stream and the difference in worst-case delivery latency along the two (or more) Member Streams' paths.

C.10 Use of the Individual recovery function

The Individual recovery function is described in 7.5, and a sample network employing the Individual recovery function is illustrated in Figure 7-3. The primary use of the Individual recovery function is to prevent stale data from being transmitted in the event that one of the Member Streams of a Compound Stream fails such that the same packet is sent over and over again. Imagine, in Figure 7-3, that the leftmost system fails on its upper port, resending the packet with sequence_number 5 over and over again on Member Stream 1. The Sequence recovery functions (7.4.2) in the two systems receiving the Member Streams 1 and 2 will discard the repeated packets until the sequence_number subparameter on the good Member Stream 2 wraps around after 65 536 packets. Then, whichever packet 5 is received first will be relayed to the next stage. It could be the new, good, Member Stream 2's packet 5 or the old, bad, Member Stream 1's packet 5. If Individual recovery functions are configured for Member Stream 1 as shown by the triangles in the two systems receiving that Stream, then all of the packet 5s after the first will be discarded there, and never reach the Sequence recovery functions.

C.11 Use of autoconfiguration

Every last detail of the operation of FRER on each individual Stream can be configured by using the managed objects in Clause 9 and Clause 10. Often, however, there is enough regularity in the use of FRER across various Streams in a network that Autoconfiguration (7.11) can be used. There are four steps to using Autoconfiguration in a given relay system or end system, as follows:

a) Deciding on which port(s) packets belonging to Member Streams will be received and transmitted, using what translations are required that do not use active Stream identification functions (C.11.1).
b) Deciding which packets can trigger Autoconfiguration (C.11.2).
c) Deciding which method for encoding the sequence_number subparameter will be used for input and output on each port (C.11.3).
d) Deciding on whether Individual recovery functions, Sequence recovery functions, or both, are to be automatically instantiated, and what controlling parameters are to be used for each instantiation (C.11.4).

NOTE—Although the following examples describe interoperation between HSR/PRP and the R-TAG, such interoperability is not the primary reason for defining autoconfiguration. Autoconfiguration greatly simplifies the amount of configuration required for networks that use only the R-TAG.

C.11.1 Routing and labeling Member Streams

Figure C-15 illustrates an example network with four Member Streams constituting a Compound Stream. This example is similar to that illustrated in Figure 7-3.

In general, the selection of paths taken by Member Streams through the network is not determined by FRER, but by other standards. Autoconfiguration does not provide a means of establishing active Stream identification functions to perform per-Stream identification transformations. In Figure C-15, let us assume for the sake of example that Member Stream 1 and Member Stream 2 both have the same destination MAC address, but are on two special VLANs 1000 and 1001, used exclusively in this network for fully managed pinned-down paths. That is, these two VLANs are not controlled by the topology protocols defined in IEEE Std 802.1Q. Let us further suppose that Member Stream 3 and Member Stream 4 are similar, but both use the same VLANs 2000. (Presumably, source address learning is disabled on this VLAN.)

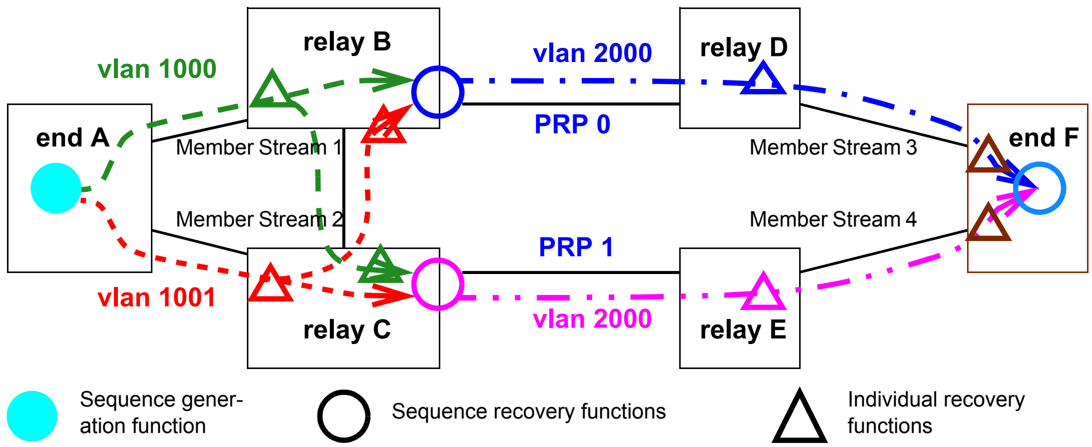

Figure C-15—Autoconfiguration example

Looking at relay system B in Figure C-15, we can see that all packets in Member Stream 3 are to be output on VLAN 2000 with a PRP sequence trailer specifying LanId 0. That means that packets from Member Streams 1 and 2 that are accepted by the Sequence recovery function in relay system B must be translated from VLAN 1000 or 1001 to VLAN 2000. The Autoconfiguration facility of 7.11 does not automatically setup or change vlan_identifiers. Relay system B must be configured to perform a VLAN ID translation on its right-hand port, translating both VLAN 1000 and VLAN 1001 to VLAN 2000 on output. If there are a great many Compound Streams, then presumably they would share the same VLAN assignments and thus make use of the same VLAN translations. IEEE 802.1Q bridges have such a VLAN ID translation capability.

Although not shown in Figure C-15, we can imagine that there are Streams with Talkers in the right side of the figure and listeners on the left. Additional VLAN mapping would then be required on the left and bottom ports of relay system B to map VLAN 2000 to VLAN 1000 and VLAN 1001. If the VLAN situation were significantly more complex, then the network might not be suitable for Autoconfiguration.

C.11.2 Recognizing packets that trigger autoconfiguration

In the example in Figure C-15, let us consider only relay system B, and only packets that follow the same routes as Member Streams 1 and 2. In that case, we expect relay system B to receive packets suitable for Autoconfiguration only on its left and bottom ports, all encoded with the R-TAG (7.8). Thus, an administrator would construct a frerAutSeqEntry (10.7.1.1) in the Sequence autoconfiguration table specifying that tag on those ports, for VLANs 1000 and 1001. This entry will, when a matching packet is received, create a tsnStreamIdEntry (9.1.1) in the Sequence identification table (10.5) using Source MAC and VLAN Stream identification (6.5). Since the R-TAG contains no indication as to which Member Stream a packet belongs, the entry is tied to the port and VLAN on which the packet was received. In this way, all of the Member Streams from end system A that take the path of Member Stream 1 are assigned one stream_handle, and all those following the path of Member Stream 2 get a second stream_handle.

Again, we can imagine that there are Streams with Talkers in the right side of Figure C-15 and listeners on the left. A second frerAutSeqEntry is required in the Sequence autoconfiguration table is required to recognize, for Autoconfiguration, the packets coming in on relay system B's right-hand port, and create entries in the Sequence identification table. This single entry specifies packets encoded with the PRP sequence trailer (see a further explanation of this in C.11.3), and includes all PRP ports—in this example, just the one. A tsnStreamIdEntry created for HSR or PRP by this entry that assigns packets a stream_handle would not be tied to the port and VLAN on which the packet was received, but to all ports covered by the tsnStreamIdEntry, and to the LanId or PathId contained in the packet. That is, the contents of the HSR sequence tag or the PRP sequence trailer override the port number.

C.11.3 Per-port packet decoding and encoding

The example in Figure C-15 also illustrates a second use of Autoconfiguration. In this network, everything to the right of relay systems B and C, including the rightmost ports on relay systems B and C, use the PRP sequence trailer (7.10), whereas all ports on relay systems A, B, and C to the left use the R-TAG (7.8). Packets transmitted to the right by relay system B, and all packets transmitted by system D and the upper port of system F are marked as being on LanId 0, and by other ports on the right side of the figure as being on LanId 1. Packets transmitted by any other port on relay systems B and C, or by system A, use the R-TAG. These distinctions are made using the Output autoconfiguration table (10.7.2). Using this table, each port is assigned an output encapsulation, including the LanId or PathId for PRP or HSR, respectively. Again, if the usage is too complex (e.g., one Stream is LanId 0 on a port, while another is LanId 1 on that same port), Autoconfiguration cannot be used.

Note that the administrator could configure the Sequence autoconfiguration tables and Output autoconfiguration tables in relay systems D and E to translate among the R-TAG, the HSR sequence tag, and the PRP sequence trailer for the purpose of interworking, even if no Individual recovery functions or Sequence recovery functions were desired in those relay systems.

C.11.4 Individual and Sequence recovery functions

Given the explanations in C.11.1, C.11.2, and C.11.3, the reader can consult 7.11.2 to understand how Individual recovery functions and Sequence recovery functions are configured as a result of Autoconfiguration. They are set up, along with the input (passive) Stream identification functions, by entries in the Sequence autoconfiguration table (10.7.1). The reader may find the following facts helpful:

a) At most one instance of the Individual recovery function is instantiated for each stream_handle associated with an autocreated tsnStreamIdEntry (9.1.1).
a) One instance of the Sequence recovery function is instantiated on each expected output port (frerAutSeqRecoveryPortList, 10.7.1.1.5) for each Compound Stream.
b) Compound Streams are inferred by associating Member Streams with the same Source MAC address and VLAN ID.

Annex D

(informative)

Bibliography

Bibliographical references are resources that provide additional or helpful material but do not need to be understood or used to implement this standard. Reference to these resources is made for informational use only.

[B1] IEEE Std 802.1BA™-2011, IEEE Standard for Local and Metropolitan Area Networks—Audio Video Bridging (AVB) Systems.[12, 13]

[B2] IEEE Std 802.3™, IEEE Standard for Ethernet.

[B3] IEEE Std 1722™-2016, IEEE Standard for a Transport Protocol for Time-Sensitive Applications in Bridged Local Area Networks.

[B4] IETF RFC 3985, Bryant, S., Ed., and P. Pate, Ed., Pseudo Wire Emulation Edge-to-Edge (PWE3) Architecture, DOI 10.17487/RFC3985, March 2005.[14]

[12]The IEEE standards or products referred to in Annex D are trademarks owned by The Institute of Electrical and Electronics Engineers, Incorporated.

[13]IEEE publications are available from The Institute of Electrical and Electronics Engineers (http://standards.ieee.org).

[14]IETF documents (i.e., RFCs) are available for download at http://www.rfc-archive.org/.